UniMart Study Guide

High-level revision notes for the current codebase, grouped by UI, backend logic, database connection, and security.

Purpose of the app: UniMart is a campus marketplace web app where students can sign up, manage a profile, view listing analytics on a dashboard, and browse marketplace listings stored in Supabase.

Main architecture: the frontend is plain HTML, CSS, and JavaScript. Supabase handles authentication, database access, and row-level access control. Shared logic lives in `app.js` and `auth.js`, while each page has its own UI logic in inline script blocks.

What To Remember Fast

- **UI:** built with static HTML pages, shared CSS, and shared shell helpers.
- **Backend in this project:** mostly frontend JavaScript calling Supabase directly through the client SDK.
- **Auth:** Supabase Auth is used for sign up, sign in, OTP verification, session checks, and password changes.
- **Database:** main tables used are `users` and `listings`.
- **Security:** passwords are never stored manually in our own tables; Supabase Auth stores them securely. The app only sends them to Supabase over the auth API.
- **RLS:** Row Level Security is used so database rows are only readable/writable when policies allow it.

UI Layer

Shared UI

`styles.css`

- Defines the brand colors, typography, spacing, buttons, forms, dropdowns, cards, and responsive layout.
- Controls the app shell: sidebar, topbar, page content spacing, filter chips, and dropdown styling.
- Contains reusable classes such as `.btn`, `.form-input`, `.dropdown-menu`, `.listings-grid`, and icon helpers like `.ui-icon`.

Shared Behavior

`app.js`

- `iconMarkup(name)`: returns the SVG markup used in toast messages.
- `showToast(...)`: displays floating success, error, or info notifications.
- `populateUserShell(user)`: fills in the logged-in user's name, role, and initials across shared UI.
- `initDropdowns()`: opens and closes profile dropdown menus.
- `setActiveNav()`: highlights the current page in the sidebar.
- `initMobileSidebar()`: handles mobile sidebar open/close behavior.
- `initPage()`: shared page initializer that checks auth, prepares the shell, and wires sign-out buttons.

Backend / Application Logic

Supabase Data + Auth Layer

`auth.js`

This file acts like the app's backend service layer inside the frontend. Instead of calling a custom Node or Express server, the app calls Supabase directly.

Function	High-level purpose
`signUp()`	Creates a new Supabase Auth user and stores extra user information in metadata during sign-up.
`signIn()`	Signs a user in using email and password through Supabase Auth.
`verifyOTP()`	Verifies the email confirmation code, then upserts the user into the `users` table.
`signOut()`	Ends the Supabase session and redirects the user back to the login page.
`requireAuth()`	Checks for an active session; if none exists, the user is redirected to login.
`getUser()`	Returns the current signed-in user if a session exists.
`updateProfile()`	Updates profile info in both the `users` table and auth metadata.
`updateCampusInfo()`	Updates university, campus, and student number in the `users` table.
`updatePassword()`	Re-authenticates the user with the current password, then changes the password through Supabase Auth.
`getListingDashboard()`	Reads the signed-in seller's listings and calculates dashboard metrics, category stats, recent items, and 6-month activity.
`getMarketplaceListings()`	Loads active marketplace listings for the browse page.
`_getProfile()`	Private helper that loads the user profile from the `users` table and falls back to auth metadata if needed.
`_buildUser()`	Builds a frontend-friendly user object from the auth user.
`getUserInitials()`	Creates avatar initials from the person's full name.

Database Connection

How We Connect

- The app uses the Supabase JavaScript SDK loaded from a CDN.
- `SUPABASE_URL` and `SUPABASE_ANON_KEY` are used to create a client with `supabase.createClient(...)`.
- The client is stored in `_sb` and used for auth and database queries.
- The anon key does **not** bypass security by itself; access is still controlled by Supabase Auth and RLS policies.

Main Tables

- **`users` table:** stores profile fields such as full name, email, account type, university, campus, and student number.
- **`listings` table:** stores marketplace items including `listing_id`, `seller_id`, `title`, `description`, `price`, `category`, `condition`, `is_tradeable`, `status`, and `created_at`.
- **Relationship:** `seller_id` links a listing to the user who created it.

Authentication And Password Safety

Security

- The app uses **Supabase Auth** instead of storing passwords in a custom table.
- During sign-up and sign-in, the password is only sent to Supabase Auth methods like `signUp()` and `signInWithPassword()`.
- The app itself does **not** store plaintext passwords anywhere in `users` or `listings`.
- Password changes go through `updatePassword()`, which first re-checks the current password, then asks Supabase to update it securely.
- Because Supabase manages the auth system, password hashing and secure auth storage are handled by Supabase rather than manual code in this project.

Authorization And RLS

Access Control

- **Authentication** answers: “Who is the user?”
- **Authorization** answers: “What is the user allowed to read or change?”
- The project uses **Row Level Security (RLS)** in Supabase so access is controlled per row in the database.
- For dashboard analytics, the app reads listings filtered by the logged-in seller’s user ID, so a user only sees their own listing analytics.
- For marketplace browsing, an RLS policy can allow authenticated users to read active listings.
- RLS is important because even if someone has the frontend code and anon key, they still cannot read or write rows unless the policy allows it.

Example ideas used in this project setup: a policy to let users read their own listings for dashboard data, and a policy to let authenticated users browse rows where `status = 'active'`.

Page-By-Page Feature Breakdown

Login

`login.html`

- Redirects already logged-in users to the marketplace.
- Handles password visibility toggle.
- Validates email and password before sign-in.
- Calls `Auth.signIn()` to create a session.
- Uses `clearErrors()` and `showError()` to manage form errors.

Signup

`signup.html`

- Collects full name, university email, password, account type, and optional campus information.
- Shows password strength feedback.
- Implements an OTP verification step after sign-up.
- Uses `Auth.signUp()` and `Auth.verifyOTP()`.
- Contains helper functions like `makeToggle()`, `showStep()`, and OTP input handlers.

Profile

`profile.html`

- Loads the authenticated user and fills the profile card.
- Lets the user edit profile details, campus info, and password.
- Uses `Auth.updateProfile()`, `Auth.updateCampusInfo()`, and `Auth.updatePassword()`.
- Includes section navigation and scroll-spy behavior for profile sections.

Dashboard

`dashboard.html`

- Calls `loadDashboard(user)` after shared page initialization.
- Uses `Auth.getListingDashboard()` to fetch seller-specific listing analytics.
- Displays active listings, sold listings, active listing value, and listings created this month.
- Renders charts for monthly listing activity and top categories.
- Shows recent listings and an empty-state when no listing data exists.

Marketplace Search

`search.html`

- Loads active listings with `Auth.getMarketplaceListings()`.
- Filters the marketplace in the browser using category chips and the search box.
- Uses `renderListings()` and `renderListingCard()` to display the cards.
- Shows a clean no-results or no-listings empty-state when needed.

Listing Management Placeholder

`listings.html`

- Currently acts as a “coming soon” page for managing a seller’s own listings.
- Exists in the shared shell and is ready for future CRUD functionality.

Important Function Revision List

Functions your team should know at a glance

File	Function	Short explanation
`app.js`	`initPage()`	Main shared entry point for authenticated pages.
`app.js`	`showToast()`	Reusable success/error/info toast notifications.
`auth.js`	`signUp()`	Registers a new user with Supabase Auth.
`auth.js`	`signIn()`	Signs the user in and starts a session.
`auth.js`	`verifyOTP()`	Confirms email and inserts profile data into `users`.
`auth.js`	`requireAuth()`	Protects pages from unauthenticated access.
`auth.js`	`updatePassword()`	Safely updates a password through Supabase Auth.
`auth.js`	`getListingDashboard()`	Builds analytics data for the dashboard.
`auth.js`	`getMarketplaceListings()`	Fetches active listings for the marketplace browse page.
`dashboard.html`	`loadDashboard()`	Loads live data and fills dashboard cards and charts.
`search.html`	`renderListings()`	Applies filters and updates the visible listing grid.
`profile.html`	`clearErrors()` / `showError()`	Simple reusable form validation helpers on the page.
`signup.html`	`showStep()`	Moves the user through account creation, verification, and success states.

Good Talking Points For A Test

- The app uses a simple frontend architecture but still separates concerns by putting shared shell code in `app.js` and auth/data logic in `auth.js`.
- Supabase replaces the need for a custom backend server for this project because it provides auth, database access, and RLS security.
- Profile data is stored in the `users` table, while login credentials are handled separately by Supabase Auth.
- The dashboard is not hardcoded anymore; it computes live analytics from the seller's listings.
- The marketplace page fetches active listings and then filters them client-side for a quick user experience.
- RLS is critical because the frontend uses a public anon key, so policies enforce who can actually read or modify rows.